

SATCOM ANTENNA CONTROL SIMULATOR

Engineering Handbook ? repository-grounded edition

Repository: SaadWajih99/SATCOM_Antenna_Project

Scope: Interactive browser-based two-axis electromechanical pointing demonstrator

Evidence basis: Source inspection and pnpm test (1 file, 6 tests passed).

NOTE: The simulator deliberately abstracts high-fidelity RF, structural and electromagnetic phenomena while retaining the electromechanical behaviours most relevant to antenna pointing, control, fault detection and safety.

1. What exists today

Implemented and verified

? Next.js 16 App Router browser UI in app/page.tsx.

? Simulation engine in src/simulation/ with typed configuration/state, PID control, plant dynamics, encoder effects, delay, metrics, safety, and watchdog outputs.

? Six visible fault toggles: encoder dropout, excessive backlash, motor saturation/degradation, wind disturbance, feedback delay, and watchdog timeout.

? Live target controls, PID controls, plant controls, antenna SVG, telemetry, charts, health rows, safety state, event log, pause, reset, and emergency stop.

? Vitest test suite: 6 passing tests in tests/simulation.test.ts.

? GitHub Actions workflow in .github/workflows/deploy.yml intended to build and deploy GitHub Pages.

Not implemented / not evidenced

? RF propagation, link budget, orbital mechanics, radiation patterns, FEM, atmospheric modelling, or hardware I/O.

? A separate health-monitor or safety-manager module; those decisions currently occur in src/simulation/simulator.ts and UI-derived health values in app/page.tsx.

? A physically validated motor electromagnetic model or experimentally identified transfer function.

? A true 3D antenna model: the current visualization is an SVG projection driven by state values.

2. Repository map

text

app/page.tsx

Client dashboard and UI-to-engine wiring

app/layout.tsx

Root metadata/layout

app/globals.css	Dashboard design system and responsive styles
src/simulation/types.ts	Types, defaults, cloning, delayed measurement
src/simulation/pidController.ts	Discrete PID calculation
src/simulation/plant.ts	Axis mechanics, motor, gearbox, encoder model
src/simulation/simulator.ts	Two-axis orchestration, metrics, safety, history
tests/simulation.test.ts	Behaviour tests for controller/plant/simulator
docs/.md	Architecture, safety, testing, limitations, interview notes
.github/workflows/deploy.yml	Build/test/Pages deployment workflow
next.config.mjs	Static export configuration for Pages
package.json	pnpm scripts and dependencies

3. End-to-end data flow

text

Slider / profile

- > React params state
- > AntennaSimulator.setTargets()
- > simulator.update(dt)
- > delayed encoder feedback
- > stepAxis() for AZ and EL
- > PID error and P/I/D terms
- > motor/gearbox/friction/backlash/encoder model
- > safety override
- > state.history and metrics
- > React setSim()
- > antenna SVG, charts, telemetry, health, event log

The important caveat is that app/page.tsx also derives some display health values. The authoritative physical state and safety label come from the simulation engine, while the percentage bars are presentation calculations.

4. Simulation execution

AntennaSimulator.update(dt) clamps the requested step to 0.001?0.1 s, advances time, obtains delayed measured encoder feedback, steps both axes, applies a blocking override for emergency stop, watchdog timeout, or failed encoder, computes a safety label, appends a bounded history, calculates metrics, and records state transitions.

The engine stores up to 600 history points. The UI timer in app/page.tsx calls the engine approximately every 50 ms while running. Real elapsed time is converted to dt, so the simulation is not dependent on a perfect browser timer.

5. Control law and plant

The controller is discrete-time. The exact implementation should be read in src/simulation/pidController.ts; its output is bounded by the configured output limit and its

integral is constrained by the configured integral limit. `stepAxis()` consumes the target, feedback, axis configuration, time step, and disturbance torque, then returns updated axis state including position, velocity, command, torque, encoder status, and PID terms.

The conceptual closed loop is:

text

reference - measured feedback = error
error -> P/I/D controller -> bounded command
command -> motor response and torque
motor torque -> gearbox efficiency and ratio
net torque -> inertia/friction/backlash dynamics
true position -> encoder resolution/noise/offset/dropout
measured position -> next controller sample

This is a demonstration model, not a validated plant identification. Units and sign conventions must be taken from `types.ts`, `plant.ts`, and the tests when extending it.

6. Fault behaviour

UI fault	Model change	Observable consequence
Encoder dropout	<code>encoder.dropout = true</code>	Feedback becomes invalid/failed; engine can force zero command and FAULT.
Excessive backlash	Gearbox backlash raised to 3.5; friction is increased	Dead-zone-like tracking degradation and larger error.
Motor saturation	UI forces motor degradation floor of 48%	Reduced effective actuation and larger residual error.
Wind disturbance	Disturbance torque becomes 0.22	Persistent external load must be rejected by control effort.
Feedback delay	Configured feedback delay is enabled	Controller acts on older measured position; transient response changes.
Watchdog timeout	<code>watchdogTimeout = true</code>	Commands are zeroed and state becomes SAFE.

Emergency stop is independent of normal PID intent: `config.emergencyStop` causes command and torque to be zeroed. Reset clears the engine state and releases the UI emergency-stop flag.

7. Safety and health interpretation

The engine safety label is deterministic:

? SAFE: emergency stop or watchdog timeout.
? FAULT: encoder failure.
? DEGRADED: combined AZ/EL error greater than 20 degrees.
? WARNING: error greater than 8 degrees.
? NORMAL: otherwise.

The UI maps WARNING to its displayed DEGRADED presentation state. This is a useful distinction

to explain in an interview: the engine and display vocabulary are not identical.

Health percentage in `toDashboardState()` is a UI-derived score based mainly on RMS error and safety penalties. The subsystem rows separately use RMS error, configured degradation, backlash, and encoder fault state. These are meaningful indicators, but they are not a standards-based condition-monitoring system.

8. Metrics

`calculateMetrics()` derives values from actual bounded history:

- ? RMS error: root mean square of the combined AZ/EL error magnitude.
- ? Maximum error: largest combined error magnitude.
- ? Maximum control effort: largest absolute axis command.
- ? Steady-state error: mean of the last up-to-25 error samples.
- ? Rise time: first history point below 10% of initial error.
- ? Overshoot: maximum error minus initial error, lower bounded at zero.
- ? Settling time: first point after which all remaining errors are within the selected threshold.

These definitions are implementation-specific. In particular, the rise/settling definitions should be reviewed before presenting them as textbook step-response metrics.

9. UI guide

`app/page.tsx` is a client component. React state stores parameters, selected profile, fault flags, emergency stop, run state, and the dashboard projection. `useEffect` synchronizes UI configuration into the engine and starts/stops the simulation interval. The antenna graphic is SVG; its rotation is driven by `sim.az.position`, while target and readouts use current target/error state.

The two charts render from the engine history. The displayed AZ chart uses `targetAz` and `actualAz`; control effort uses `commandAz`. The chart range is fixed for display, so clipping is possible when values exceed the visual range.

10. Tests and reproducibility

Run:

```
bash
pnpm install
pnpm test
pnpm build
pnpm dev
```

The verified baseline in this handbook is `pnpm test`: 1 test file, 6 tests, all passed. Build and Pages deployment must be checked in the GitHub Actions run for the current commit; a failed workflow means the public Pages URL is not yet a valid demonstration endpoint.

11. Interview explanation

A concise answer is:

NOTE: ?This is a browser-based two-axis pointing simulator. A target is compared with delayed encoder feedback, each axis is passed through a discrete PID controller, and the command drives a simplified motor, gearbox, friction, backlash, inertia, and encoder model. The engine records actual history and derives tracking metrics. Fault toggles modify real model parameters or feedback validity, while deterministic safety overrides disable the drive for encoder failure, watchdog timeout, or emergency stop. I present it as an educational electromechanical demonstrator, not as a validated RF or flight-qualified model.?

12. Learning checklist

1. Trace Page -> setTargets -> update -> stepAxis -> PID -> state.history.
2. Change only Kp, then observe command and error; repeat with backlash and encoder dropout.
3. Read types.ts before changing units or defaults.
4. Read plant.ts to connect torque, inertia, friction, gearbox, and encoder effects.
5. Use the tests as executable examples of intended behaviour.
6. Treat every UI health percentage as a derived indicator and verify its source before claiming it is an engineering standard.

13. Limitations and next defensible improvements

The next engineering improvements should be incremental: separate health and safety modules, add explicit per-axis telemetry for all PID terms, expand tests for every requested fault, make units explicit in types, and add a reproducible Pages smoke test. Do not add RF or orbital detail until the electromechanical model, measurements, and safety transitions are independently verified.

Coverage status: This handbook covers every project-owned source/configuration area discovered in the repository inventory. Generated dependencies and vendor code are intentionally described by usage rather than reproduced.

Generated from the repository state inspected for this handbook; line numbers should be regenerated after future edits.